



Universidad  
de Alicante

SISTEMAS INTELIGENTES

# PRÁCTICA 1

## SATISFACCION DE RESTRICCIONES SUDOKU

FECHA DE ENTREGA: 12 DE NOVIEMBRE DE 2025

TURNO: JUEVES DE 17:00 A 19:00

GRUPO 5

PROFESOR: LUCAS MARTÍNEZ BERNABÉU

Jevgenijs Tjurins Boloveckiy  
51238455K



|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>Explicación de la clase Variable.....</b>                                          | <b>3</b>  |
| <b>Explicación de cómo se han tratado las casillas con números fijos.....</b>         | <b>3</b>  |
| <b>Especificación formal del problema.....</b>                                        | <b>4</b>  |
| La especificación formal debe detallar la tupla $\langle V, E, c, I, a \rangle$ ..... | 4         |
| V: Conjunto de variables.....                                                         | 4         |
| E: Conjunto de aristas.....                                                           | 4         |
| c: Función de correspondencia entre restricciones y variables.....                    | 5         |
| I : Definición del tipo de restricción.....                                           | 5         |
| a: Orden de Asignación de Variables.....                                              | 5         |
| Ejemplo de formalización con Sudoku 4x4.....                                          | 6         |
| V: Conjunto de variables.....                                                         | 6         |
| E: Conjunto de aristas.....                                                           | 6         |
| c: Función de correspondencia entre restricciones y variables.....                    | 7         |
| I : Definición del tipo de restricción.....                                           | 8         |
| a: Orden de Asignación de Variables.....                                              | 9         |
| <b>Apartado de experimentación.....</b>                                               | <b>10</b> |
| Sudoku con muchas celdas fijas (fácil).....                                           | 10        |
| Sudoku con pocas celdas fijas (difícil).....                                          | 11        |
| Sudoku inconsistente.....                                                             | 13        |
| Sudoku vacío.....                                                                     | 14        |
| <b>Estudio de tiempos.....</b>                                                        | <b>16</b> |
| <b>Utilización de gráficas para el análisis de tiempos.....</b>                       | <b>18</b> |
| <b>Dificultades encontradas.....</b>                                                  | <b>20</b> |
| <b>Uso de IAs generativas: explicación del uso que se ha hecho de ellas.....</b>      | <b>21</b> |
| <b>Apartado de referencias bibliográficas.....</b>                                    | <b>21</b> |

## Explicación de la clase *Variable*

La clase *Variable* representa cada una de las celdas del tablero de Sudoku dentro del CSP.

Su función principal es almacenar el valor actual de la celda y su dominio de posibles valores, permitiendo así que los algoritmos operen de forma uniforme sobre todas las celdas.

Cada instancia de esta clase corresponde a una celda vacía o a una casilla con un número fijo del tablero inicial.

Si una celda ya tiene un valor fijo en el tablero inicial, la variable se marca como **fija** (*fijo = True*) y su dominio se reduce únicamente a ese valor.

Si la celda está vacía, se considera **variable libre** y su dominio se inicializa con todos los valores posibles del 1 al 9.

En el programa principal, una vez cargado el Sudoku desde el archivo se ejecuta la función para crear las variables a partir del tablero, que recorre las 81 celdas y crea una lista de objetos *Variable*.

Cada una se inicializa con el valor actual del tablero, de manera que si el número es distinto de '0', la variable se marca como fija y su dominio contiene solo ese valor.

## Explicación de cómo se han tratado las casillas con números fijos

En la implementación, las casillas fijas se identifican al crear las variables del tablero. Si una celda tiene un número distinto de cero, se marca como fija y su dominio se limita solo a ese valor.

En **Backtracking**, estas casillas se **saltan** porque no cambian, y así conseguimos reducir el trabajo del algoritmo.

En **Forward Checking**, las casillas fijas **sí participan**, por lo que no se saltan ya que su valor ayuda a **reducir los dominios de las celdas vecinas** desde el inicio.

En **AC3**, las casillas fijas **no se procesan como variables principales por lo que se saltan** porque su dominio no puede modificarse, pero **sí aparecen como vecinas en la cola de arcos**, permitiendo que su valor fijo contribuya a **podar los dominios de las demás variables**.

## Especificación formal del problema

La especificación formal debe detallar la tupla  $\langle V, E, c, l, a \rangle$

El Sudoku puede modelarse como un **problema de satisfacción de restricciones (CSP)**, en el cual se busca asignar valores a un conjunto de variables de forma que todas las restricciones definidas entre ellas se cumplan simultáneamente.

De forma general, un CSP se representa mediante la **tupla**:

$$\langle V, E, c, l, a \rangle$$

donde cada elemento describe un componente fundamental del problema.

En el caso concreto del Sudoku, esta tupla se define del siguiente modo:

### *V: Conjunto de variables*

El conjunto **V** representa todas las variables del sudoku.

Cada *Variable* corresponde a una **celda del tablero**, por lo que existen  $9 \times 9 = 81$  variables:

$$V = \{V_1, V_2, \dots, V_{81}\}$$

Cada *Variable* representa una posición concreta del tablero y podrá tomar un valor entre 1 y 9 según las restricciones de fila, columna y bloque.

Y donde el **dominio discreto y finito** de cada variable, correspondiente a los posibles valores que puede tomar una celda:

$$D_i = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}, \quad \forall i, 1 \leq i \leq 81$$

### *E: Conjunto de aristas*

El conjunto **E** representa las **aristas del grafo de restricciones**, es decir, los pares de variables que están relacionadas entre sí dentro del problema.

En el caso del Sudoku, 2 variables estarán conectadas mediante una arista si **pertenecen a la misma fila, columna o subcuadrante 3x3**.

Cada arista, por tanto, indica que entre esas dos celdas existe una **restricción binaria** que impide que ambas tomen el mismo valor.

Y puede expresarse como:

$$E = \{e_{ij} : 1 < i < j \leq 81 \mid V_i, V_j \text{ comparten fila, columna o bloque}\}$$

Dado que cada *Variable* (celda) está conectada con **20 celdas distintas**, y el tablero contiene **81 variables**, el número total de aristas sería de:

$$m = \frac{81 \times 20}{2} = 810$$

### *c: Función de correspondencia entre restricciones y variables*

La función *c* asocia a cada restricción (arista) el par de variables que intervienen en ella.

En el caso del Sudoku, las restricciones son **binarias**, ya que se aplican sobre **pares de variables** que comparten una misma **fila, columna o subcuadrante 3×3** del tablero.

Se puede representar como:

- Si dos variables  $V_i$  y  $V_j$  pertenecen a la misma fila, columna o bloque, entonces  $V_i \neq V_j$ . es decir:

$$c : E \rightarrow V \times V, c(e_{ij}) = (V_i, V_j)$$

Estas restricciones aseguran que no existan valores repetidos en ninguna fila, columna ni región del tablero.

### *l: Definición del tipo de restricción*

Define el tipo de restricción (valores distintos). Y se representaría como:

$$l : E \rightarrow \{\text{distinto valor}\}, \text{restricción de desigualdad entre celdas relacionadas}$$

Y para establecer la relación de consistencia binaria, es decir que 2 celdas conectadas no puedan tener el mismo valor lo formalizamos como:

$$\rho((V_i, V_j)) = \{(v_i, v_j) \in D_i \times D_j : v_i \neq v_j\}$$

### *a: Orden de Asignación de Variables*

Se entiende como el orden en que se prueban las asignaciones, útil para el backtracking:

$$a : \text{orden de asignación de variables en } [1, 81]$$

## Ejemplo de formalización con Sudoku 4x4

### *V: Conjunto de variables*

Usamos 16 variables, una por celda, en orden fila-mayor:

Índices  $\leftrightarrow$  posiciones:

1=(1,1), 2=(1,2), 3=(1,3), 4=(1,4)

5=(2,1), 6=(2,2), 7=(2,3), 8=(2,4)

9=(3,1), 10=(3,2), 11=(3,3), 12=(3,4)

13=(4,1), 14=(4,2), 15=(4,3), 16=(4,4)

$$V = \{V_1, V_2, V_3, V_4, V_5, V_6, V_7, V_8, V_9, V_{10}, V_{11}, V_{12}, V_{13}, V_{14}, V_{15}, V_{16}\}$$

Dominio:

$$D_i = \{1, 2, 3, 4\}, \forall i, 1 \leq i \leq 16$$

(Si alguna celda fuera fija, su dominio sería ese único valor)

### *E: Conjunto de aristas*

Dos variables se conectan si comparten **fila**, **columna** o **bloque 2x2**.

Para no repetir, listamos:

- i. todas las **filas** (24 pares)
- ii. todas las **columnas** (24 pares)
- iii. los **pares diagonales de bloque** (8 pares) que no están ya en filas/columnas

Total **56** aristas.

| Fila                                                                                                                                                   | Columna                                                                                                                                                        | Bloque                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>F1 (1,2,3,4):</b><br>e <sub>1</sub> = (1,2)<br>e <sub>2</sub> = (1,3)<br>e <sub>3</sub> = (1,4)<br>e <sub>4</sub> = (2,3)<br>e <sub>5</sub> = (2,4) | <b>C1 (1,5,9,13):</b><br>e <sub>25</sub> = (1,5)<br>e <sub>26</sub> = (1,9)<br>e <sub>27</sub> = (1,13)<br>e <sub>28</sub> = (5,9)<br>e <sub>29</sub> = (5,13) | <b>B1 (1,2,5,6):</b><br>e <sub>49</sub> = (1,6)<br>e <sub>50</sub> = (2,5)<br><br><b>B2 (3,4,7,8):</b><br>e <sub>51</sub> = (3,8) |

|                                                                                                                                                                     |                                                                                                                                                                                          |                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $e_6 = (3,4)$<br><br><b>F2 (5,6,7,8):</b><br>$e_7 = (5,6)$<br>$e_8 = (5,7)$<br>$e_9 = (5,8)$<br>$e_{10} = (6,7)$<br>$e_{11} = (6,8)$<br>$e_{12} = (7,8)$<br><br>... | $e_{30} = (9,13)$<br><br><b>C2 (2,6,10,14):</b><br>$e_{31} = (2,6)$<br>$e_{32} = (2,10)$<br>$e_{33} = (2,14)$<br>$e_{34} = (6,10)$<br>$e_{35} = (6,14)$<br>$e_{36} = (10,14)$<br><br>... | $e_{52} = (4,7)$<br><br><b>B3 (9,10,13,14):</b><br>$e_{53} = (9,14)$<br>$e_{54} = (10,13)$<br><br><b>B4 (11,12,15,16):</b><br>$e_{55} = (11,16)$<br>$e_{56} = (12,15)$ |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

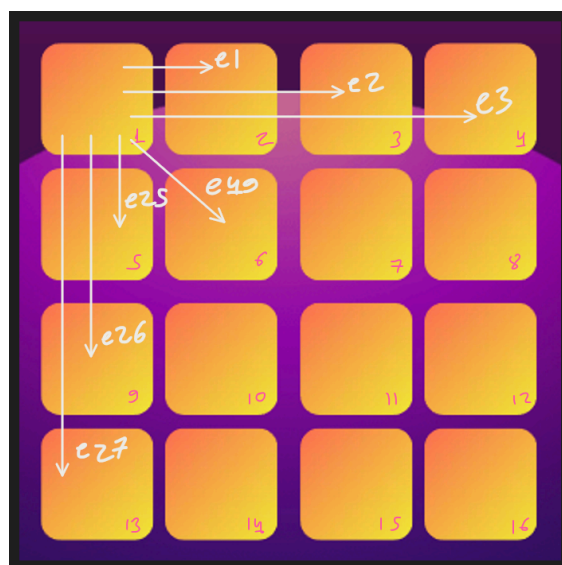


Figura 01: Relación Aristas Sudoku 4x4

c: Función de correspondencia entre restricciones y variables

| Fila                                                                                                                                                                                                                                                             | Columna                                                                                                                                                                                                                                                                                                       | Bloque                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fila 1 (1,2,3,4):</b><br>$c(e_1) = (V_1, V_2)$<br>$c(e_2) = (V_1, V_3)$<br>$c(e_3) = (V_1, V_4)$<br>$c(e_4) = (V_2, V_3)$<br>$c(e_5) = (V_2, V_4)$<br>$c(e_6) = (V_3, V_4)$<br><br><b>Fila 2 (5,6,7,8):</b><br>$c(e_7) = (V_5, V_6)$<br>$c(e_8) = (V_5, V_7)$ | <b>Columna 1 (1,5,9,13):</b><br>$c(e_{25}) = (V_1, V_5)$<br>$c(e_{26}) = (V_1, V_9)$<br>$c(e_{27}) = (V_1, V_{13})$<br>$c(e_{28}) = (V_5, V_9)$<br>$c(e_{29}) = (V_5, V_{13})$<br>$c(e_{30}) = (V_9, V_{13})$<br><br><b>Columna 2 (2,6,10,14):</b><br>$c(e_{31}) = (V_2, V_6)$<br>$c(e_{32}) = (V_2, V_{10})$ | <b>Bloque 1 (1,2,5,6):</b><br>$c(e_{49}) = (V_1, V_6)$<br>$c(e_{50}) = (V_2, V_5)$<br><br><b>Bloque 2 (3,4,7,8):</b><br>$c(e_{51}) = (V_3, V_8)$<br>$c(e_{52}) = (V_4, V_7)$<br><br><b>Bloque 3 (9,10,13,14):</b><br>$c(e_{53}) = (V_9, V_{14})$ |

|                                                                                                                  |                                                                                                                                    |                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| $c(e_9) = (V_5, V_8)$<br>$c(e_{10}) = (V_6, V_7)$<br>$c(e_{11}) = (V_6, V_8)$<br>$c(e_{12}) = (V_7, V_8)$<br>... | $c(e_{33}) = (V_2, V_{14})$<br>$c(e_{34}) = (V_6, V_{10})$<br>$c(e_{35}) = (V_6, V_{14})$<br>$c(e_{36}) = (V_{10}, V_{14})$<br>... | $c(e_{54}) = (V_{10}, V_{13})$<br><br><b>Bloque 4 (11,12,15,16):</b><br>$c(e_{55}) = (V_{11}, V_{16})$<br>$c(e_{56}) = (V_{12}, V_{15})$ |
|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|

### *l : Definición del tipo de restricción*

Como todos los dominios son  $D_i = \{1, 2, 3, 4\}$ , la relación de **desigualdad** es la misma para **todas** las aristas:

$$S_{\neq} = \{(1, 2), (1, 3), (1, 4), (2, 1), (2, 3), (2, 4), (3, 1), (3, 2), (3, 4), (4, 1), (4, 2), (4, 3)\}$$

Es decir, que para la primera coordenada:

- 1 puede ir con 2, 3, 4
- 2 puede ir con 1, 3,
- 3 con 1, 2, 4
- 4, con 1, 2, 3

Por lo que no aparecen pares iguales como (1, 1), (2, 2), (3, 3), (4, 4).

Y para cada arista  $e_k$  (de la lista de 56 aristas que ya enumeramos), se fija:

- $l(e_1) = \text{"distinto valor"}$  (es decir,  $v_i \neq v_j$  en esa arista)
- $l(e_2) = \text{"distinto valor"}$
- ...
- $l(e_{56}) = \text{"distinto valor"}$

Luego, **cada** arista  $e_k$  que conecta  $(V_a, V_b)$ , la relación binaria de consistencia es **exactamente el mismo conjunto de pares numéricos**  $S_{\neq}$  definido arriba:

- $\rho(e_1) = S_{\neq}$
- $\rho(e_2) = S_{\neq}$
- ...
- $\rho(e_{56}) = S_{\neq}$

Ahora bien, si alguna celda estuviera **fijada** (p. ej.,  $D_i = \{1\}$ ), entonces para cualquier arista que involucre a  $V_i$  la relación  $\rho$  se **restringe** automáticamente a los pares de  $S_{\neq}$  cuya primera



(o segunda) coordenada sea ese valor fijo, es decir, que se pueden quitar los pares que ya no son posibles.

*a: Orden de Asignación de Variables*

Si no tenemos en cuenta ninguna heurística (MRV o Heurístico de grado, etc.) un ejemplo simple sería usar el orden natural:

$a=(1,2,3,4, 5,6,7,8, 9,10,11,12, 13,14,15,16)$

## Apartado de experimentación

En la que se describan las diferentes pruebas realizadas a los algoritmos dejando bien claro el objetivo de las pruebas: plantillas con muchas celdas fijas, con pocas, etc.

### *Sudoku con muchas celdas fijas (fácil)*

El objetivo de estas pruebas es **evaluar el comportamiento de los algoritmos Backtracking (BK), Forward Checking (FC) y AC3** en sudokus con alto grado de restricción inicial, es decir, **tableros con muchas casillas fijas y bajo espacio de búsqueda**.

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 5 | 3 | 4 | 6 | 7 | 8 | 9 | 1 | 2 |
| 6 | 7 | 2 | 1 | 9 | 5 | 3 | 4 | 8 |
| 1 | 9 | 8 | 3 | 4 | 2 | 5 | 6 | 7 |
| 8 | 5 | 9 | 7 | 6 | 1 | 4 | 2 | 3 |
| 4 | 2 | 6 | 8 | 5 | 3 | 7 | 9 | 1 |
| 7 | 1 | 3 | 9 | 2 | 4 | 8 | 5 | 6 |
| 9 | 6 | 1 | 5 | 3 | 7 | 2 | 8 | 4 |
| 2 | 8 | 7 | 4 | 1 | 9 | 6 | 3 | 5 |
| 3 | 4 | 5 | 2 | 8 | 6 | 1 | 7 | 9 |

Figura 02: 'm1.txt' resuelto

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 6 | 4 | 5 | 7 | 9 | 3 | 2 | 8 |
| 3 | 2 | 9 | 1 | 8 | 6 | 4 | 7 | 5 |
| 8 | 7 | 5 | 4 | 3 | 2 | 1 | 9 | 6 |
| 9 | 3 | 2 | 8 | 6 | 7 | 5 | 4 | 1 |
| 7 | 5 | 6 | 3 | 4 | 1 | 2 | 8 | 9 |
| 4 | 1 | 8 | 2 | 9 | 5 | 7 | 6 | 3 |
| 6 | 4 | 3 | 7 | 5 | 8 | 9 | 1 | 2 |
| 5 | 9 | 1 | 6 | 2 | 4 | 8 | 3 | 7 |
| 2 | 8 | 7 | 9 | 1 | 3 | 6 | 5 | 4 |

Figura 03: 'm2.txt' resuelto

m1 Sin recorte AC3:

- BK -> Recursiones: 6428 | Asignaciones: 37652 | Tiempo: 0.01759820000006584s
- FC -> Recursiones: 3268 | Asignaciones: 6156 | Tiempo: 0.00979879999977129s

m1 Con recorte AC3<sup>1</sup>:

- BK -> Recursiones: 82 | Asignaciones: 51 | Tiempo: 6.820000089646783e-05s
- FC -> Recursiones: 82 | Asignaciones: 81 | Tiempo: 0.0001274999995075632s

m2 Sin recorte AC3:

---

<sup>1</sup> En este estudio de tiempos se ha considerado únicamente la ejecución de los algoritmos BK y FC, excluyendo el tiempo empleado por AC3 en la fase de preprocesamiento con el objetivo de evaluar de forma más precisa la eficiencia de los métodos de búsqueda en el proceso de resolución del Sudoku, sin que el tiempo de reducción de dominios afecte a la comparación directa entre ambos algoritmos.

- BK -> Recursiones: 494 | Asignaciones: 1010 | Tiempo: 0.0012858000009146053s
- FC -> Recursiones: 435 | Asignaciones: 546 | Tiempo: 0.000677699999869219s

m2 Con recorte AC3<sup>1</sup>:

- BK -> Recursiones: 494 | Asignaciones: 255 | Tiempo: 0.000213699999221717s
- FC -> Recursiones: 225 | Asignaciones: 235 | Tiempo: 0.00032439999995403923s

El espacio de búsqueda es reducido, por lo que **todos los algoritmos resuelven el problema rápidamente**.

- El **Backtracking** encuentra la solución sin necesidad de muchas recursiones.
- El **Forward Checking** apenas realiza podas adicionales, ya que las restricciones iniciales son suficientes para guiar la búsqueda.
- **AC3** no aporta una mejora significativa, pero resuelve el sudoku como veremos más adelante.

Los resultados muestran que, en problemas muy restringidos, **todos los algoritmos son altamente eficientes**, resolviendo el Sudoku en tiempos casi instantáneos.

En conclusión, parece que **AC3 y/o FC resultarán más útiles en problemas menos restringidos**, donde puede realizar una poda inicial significativa, mientras que en este tipo de instancias su efecto parece marginal.

### *Sudoku con pocas celdas fijas (difícil)*

El objetivo de estas pruebas es **analizar el comportamiento de los algoritmos BK, FC y AC3 en sudokus con pocas celdas fijas**, es decir, con **bajo nivel de restricción inicial**.

Se pretende comprobar hasta qué punto el preprocesamiento mediante **AC3** y la **poda anticipada de FC** pueden reducir el número de recursiones y el tiempo de ejecución frente al Backtracking simple.

Para este propósito, se han seleccionado 2 sudokus que requieren un mayor número de recursiones para su resolución. El primero de ellos (*m3.txt*) fue obtenido del sitio web [Sudoku Solver by Andrew Stuart](#), mientras que el segundo (*m4.txt*) fue generado mediante la inteligencia artificial generativa *ChatGPT*.

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 9 | 8 | 5 | 6 | 7 | 2 | 1 | 3 | 4 |
| 3 | 4 | 7 | 8 | 1 | 5 | 9 | 6 | 2 |
| 2 | 6 | 1 | 3 | 9 | 4 | 5 | 8 | 7 |
| 7 | 1 | 6 | 5 | 2 | 3 | 8 | 4 | 9 |
| 8 | 9 | 3 | 7 | 4 | 6 | 2 | 1 | 5 |
| 5 | 2 | 4 | 9 | 8 | 1 | 3 | 7 | 6 |
| 6 | 5 | 2 | 1 | 3 | 7 | 4 | 9 | 8 |
| 4 | 3 | 9 | 2 | 6 | 8 | 7 | 5 | 1 |
| 1 | 7 | 8 | 4 | 5 | 9 | 6 | 2 | 3 |

Figura 04: 'm3.txt' resuelto

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 8 | 5 | 9 | 6 | 1 | 2 | 4 | 3 | 7 |
| 7 | 2 | 3 | 8 | 5 | 4 | 1 | 6 | 9 |
| 1 | 6 | 4 | 3 | 7 | 9 | 5 | 2 | 8 |
| 9 | 8 | 6 | 1 | 4 | 7 | 3 | 5 | 2 |
| 3 | 7 | 5 | 2 | 6 | 8 | 9 | 1 | 4 |
| 2 | 4 | 1 | 5 | 9 | 3 | 7 | 8 | 6 |
| 4 | 3 | 2 | 9 | 8 | 1 | 6 | 7 | 5 |
| 6 | 1 | 7 | 4 | 2 | 5 | 8 | 9 | 3 |
| 5 | 9 | 8 | 7 | 3 | 6 | 2 | 4 | 1 |

Figura 05: 'm4.txt' resuelto

m3 Sin recorte AC3:

- BK -> Recursiones: 618178 | Asignaciones: 4881618 | Tiempo: 2.21827139999991s
- FC -> Recursiones: 456815 | Asignaciones: 869940 | Tiempo: 1.278351999999935s

m3 Con recorte AC3<sup>1</sup>:

- BK -> Recursiones: 618178 | Asignaciones: 2214614 | Tiempo: 1.12364420000008s
- FC -> Recursiones: 26991 | Asignaciones: 40792 | Tiempo: 0.0527203999999088s

m4 Sin recorte AC3:

- BK -> Recursiones: 461051 | Asignaciones: 3020499 | Tiempo: 1.33270179999999s
- FC -> Recursiones: 440621 | Asignaciones: 913707 | Tiempo: 1.353084500000048s

m4 Con recorte AC3<sup>1</sup>:

- BK -> Recursiones: 461051 | Asignaciones: 1312710 | Tiempo: 0.70012810000012s
- FC -> Recursiones: 164755 | Asignaciones: 211130 | Tiempo: 0.302521200000114s

En estas instancias, el tablero contiene un número reducido de valores iniciales, lo que incrementa enormemente el espacio de búsqueda.

- El **Backtracking puro** experimenta un número muy elevado de recursiones y asignaciones antes de encontrar la solución.
- El **Forward Checking** mejora considerablemente el rendimiento, ya que elimina valores inconsistentes de los dominios antes de continuar la búsqueda.

- La aplicación previa de **AC3** permite reducir aún más los dominios iniciales, disminuyendo la cantidad de caminos explorados por BK y FC, y donde vemos que FC es el que más beneficiado sale en gracias al primer podado de AC3.

En conjunto, estas pruebas demuestran que en instancias difíciles, con gran espacio de búsqueda, la **combinación de AC3 y FC es la estrategia más eficiente**, logrando resolver el problema de forma mucho más rápida y con un número muy inferior de operaciones respecto a Backtracking.

### *Sudoku inconsistente*

El objetivo de esta prueba es **analizar el comportamiento de los algoritmos BK, FC y AC3 frente a una plantilla inconsistente**, es decir, un Sudoku que **no posee solución válida** debido a conflictos presentes desde el inicio (por ejemplo, números repetidos en una misma fila, columna o subcuadro).

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 8 | 5 |   |   |   | 2 | 4 |   |   |
| 7 | 2 |   |   |   |   |   |   | 9 |
|   |   | 4 |   |   |   |   |   |   |
|   |   |   | 1 |   | 7 |   |   | 2 |
| 3 |   | 5 |   |   |   | 9 |   |   |
|   | 4 |   |   |   |   |   |   |   |
|   |   |   |   | 8 |   |   | 7 |   |
|   | 1 | 7 |   |   |   |   |   |   |
|   |   |   |   | 3 | 6 |   | 4 | 4 |

Figura 06: 'm5.txt' no resuelto por inconsistencia

m5 Sin recorte AC3:

- BK -> Recursiones: 387083 | Asignaciones: 2561355 | Tiempo: 1.1443398000010s
- FC -> Recursiones: 381411 | Asignaciones: 862833 | Tiempo: 1.33968090000053s

m5 Con recorte AC3<sup>1</sup>:

- BK -> Recursiones: 387083 | Asignaciones: 1058954 | Tiempo: 0.5524635000002s
- FC -> Recursiones: 158061 | Asignaciones: 201589 | Tiempo: 0.28241550000166s

En este caso:

- Los algoritmos de **Backtracking (BK)** y **Forward Checking (FC)** son finalmente los que detectan la falta de solución, al encontrarse con un punto en el que **ninguna variable puede ser asignada sin violar las restricciones**.
- Se observa que el **preprocesamiento con AC3 reduce el espacio de búsqueda** (al eliminar valores imposibles de algunos dominios), lo que se traduce en una **disminución notable del tiempo de ejecución** respecto a las versiones sin recorte.

La prueba *m5* demuestra que AC3 **no garantiza la detección de inconsistencias globales**, pero **sí mejora la eficiencia** de los algoritmos de búsqueda que se ejecutan después. En problemas sin solución, solo **BK** y **FC** pueden confirmar la inconsistencia mediante la exploración completa del espacio de búsqueda.

### Sudoku vacío

El objetivo de esta prueba es **analizar el comportamiento de los algoritmos BK, FC y AC3 en un escenario sin información inicial para analizar rendimiento base y la eficiencia intrínseca**, donde el Sudoku se encuentra completamente vacío.

En este contexto, todas las variables parten con dominios completos (1–9) y no existen restricciones que orienten la búsqueda.

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 4 | 5 | 6 | 7 | 8 | 9 | 1 | 2 | 3 |
| 7 | 8 | 9 | 1 | 2 | 3 | 4 | 5 | 6 |
| 2 | 1 | 4 | 3 | 6 | 5 | 8 | 9 | 7 |
| 3 | 6 | 5 | 8 | 9 | 7 | 2 | 1 | 4 |
| 8 | 9 | 7 | 2 | 1 | 4 | 3 | 6 | 5 |
| 5 | 3 | 1 | 6 | 4 | 2 | 9 | 7 | 8 |
| 6 | 4 | 2 | 9 | 7 | 8 | 5 | 3 | 1 |
| 9 | 7 | 8 | 5 | 3 | 1 | 6 | 4 | 2 |

Figura 07: 'm6.txt' Resuelto

m6 Sin recorte AC3:

- BK -> Recursiones: 392 | Asignaciones: 3195 | Tiempo: 0.0020688000004156493s
- FC -> Recursiones: 194 | Asignaciones: 280 | Tiempo: 0.000909399997908622s

m6 Con recorte AC3<sup>1</sup>:

- BK -> Recursiones: 392 | Asignaciones: 3195 | Tiempo: 0.001891699997941032s
- FC -> Recursiones: 194 | Asignaciones: 280 | Tiempo: 0.0008103000000119209s

En este caso:

- El **Backtracking** no enfrenta un alto nivel de dificultad, ya que no existen restricciones que generen conflictos entre variables. Aunque debe recorrer múltiples combinaciones teóricas, la ausencia de contradicciones hace que la resolución sea **rápida y directa**.
- El **Forward Checking** apenas realiza podas, puesto que no hay restricciones activas que eliminar. Sin embargo, mantiene un rendimiento ligeramente mejor que el Backtracking al evitar algunas asignaciones redundantes.
- El **AC3**, en este escenario, **no aporta ninguna mejora**, ya que todos los dominios iniciales son iguales y no existe información que propagar. Su efecto es prácticamente nulo tanto en reducción de dominios como en tiempo.

Por lo que el Sudoku vacío resulta ser la **instancia más sencilla** para el sistema, no por tener pocas restricciones, sino porque carece completamente de ellas. Esto hace que los algoritmos trabajen sin conflictos y alcancen tiempos de ejecución mínimos.

## Estudio de tiempos

Determinar si la aplicación de AC3 reduce los tiempos de ejecución de los otros algoritmos con distintos sudokus

| Sudoku | AC3 (s) | BK sin AC3 (s) | BK con AC3 (s) | FC sin AC3 (s) | FC con AC3 (s) | Comentario principal                          |
|--------|---------|----------------|----------------|----------------|----------------|-----------------------------------------------|
| m1     | 0,0031  | 0,0168         | 0,0001         | 0,0102         | 0,0001         | AC3 apenas necesario pero acelera ligeramente |
| m2     | 0,0009  | 0,0007         | 0,0002         | 0,0008         | 0,0003         | AC3 impacto moderado                          |
| m3     | 0,0029  | 2,1506         | 1,1059         | 1,2884         | 0,0567         | AC3 y FC reducen drásticamente el tiempo      |
| m4     | 0,0032  | 1,3182         | 0,6747         | 1,3607         | 0,3105         | AC3 mejora el rendimiento de ambos algoritmos |
| m5     | 0,0028  | 1,1124         | 0,5148         | 1,3003         | 0,2893         | AC3 no detecta el error pero acelera BK/FC    |
| m6     | 0,0012  | 0,0013         | 0,0013         | 0,0006         | 0,0006         | Sin mejora AC3                                |

Como podemos apreciar en la tabla los resultados muestran que **Forward Checking (FC)** es, en general, **más eficiente que Backtracking (BK)**, aunque su rendimiento depende del grado de restricción del Sudoku:

- En los casos **fáciles o muy restringidos (m1, m2)**, FC apenas mejora a BK, ya que el problema está casi resuelto y la verificación adicional solo añade coste, pero en estos casos como m1 incluso llega a resolver el sudoku debido la gran poda que realiza:



| DOMINIOS ANTES DEL AC3                | DOMINIOS DESPUÉS DEL AC3 |                                       |               |
|---------------------------------------|--------------------------|---------------------------------------|---------------|
| 00 Dominio: 5                         | 00 Dominio: 5            | 44 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 44 Dominio: 5 |
| 01 Dominio: 3                         | 01 Dominio: 3            | 45 Dominio: 3                         | 45 Dominio: 3 |
| 02 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 02 Dominio: 4            | 46 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 46 Dominio: 7 |
| 03 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 03 Dominio: 6            | 47 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 47 Dominio: 9 |
| 04 Dominio: 7                         | 04 Dominio: 7            | 48 Dominio: 1                         | 48 Dominio: 1 |
| 05 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 05 Dominio: 8            | 50 Dominio: 7                         | 50 Dominio: 7 |
| 06 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 06 Dominio: 9            | 51 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 51 Dominio: 1 |
| 07 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 07 Dominio: 1            | 52 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 52 Dominio: 3 |
| 08 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 08 Dominio: 2            | 53 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 53 Dominio: 9 |
| 10 Dominio: 6                         | 10 Dominio: 6            | 54 Dominio: 2                         | 54 Dominio: 2 |
| 11 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 11 Dominio: 7            | 55 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 55 Dominio: 4 |
| 12 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 12 Dominio: 2            | 56 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 56 Dominio: 8 |
| 13 Dominio: 1                         | 13 Dominio: 1            | 57 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 57 Dominio: 5 |
| 14 Dominio: 9                         | 14 Dominio: 9            | 58 Dominio: 6                         | 58 Dominio: 6 |
| 15 Dominio: 5                         | 15 Dominio: 5            | 60 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 60 Dominio: 9 |
| 16 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 16 Dominio: 3            | 61 Dominio: 6                         | 61 Dominio: 6 |
| 17 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 17 Dominio: 4            | 62 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 62 Dominio: 1 |
| 18 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 18 Dominio: 8            | 63 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 63 Dominio: 5 |
| 20 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 20 Dominio: 1            | 64 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 64 Dominio: 3 |
| 21 Dominio: 9                         | 21 Dominio: 9            | 65 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 65 Dominio: 7 |
| 22 Dominio: 8                         | 22 Dominio: 8            | 66 Dominio: 2                         | 66 Dominio: 2 |
| 23 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 23 Dominio: 3            | 67 Dominio: 8                         | 67 Dominio: 8 |
| 24 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 24 Dominio: 4            | 68 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 68 Dominio: 4 |
| 25 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 25 Dominio: 2            | 70 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 70 Dominio: 2 |
| 26 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 26 Dominio: 5            | 71 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 71 Dominio: 8 |
| 27 Dominio: 6                         | 27 Dominio: 6            | 72 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 72 Dominio: 7 |
| 28 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 28 Dominio: 7            | 73 Dominio: 4                         | 73 Dominio: 4 |
| 30 Dominio: 8                         | 30 Dominio: 8            | 74 Dominio: 1                         | 74 Dominio: 1 |
| 31 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 31 Dominio: 5            | 75 Dominio: 9                         | 75 Dominio: 9 |
| 32 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 32 Dominio: 9            | 76 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 76 Dominio: 6 |
| 33 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 33 Dominio: 7            | 77 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 77 Dominio: 3 |
| 34 Dominio: 6                         | 34 Dominio: 6            | 78 Dominio: 5                         | 78 Dominio: 5 |
| 35 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 35 Dominio: 1            | 80 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 80 Dominio: 3 |
| 36 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 36 Dominio: 4            | 81 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 81 Dominio: 4 |
| 37 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 37 Dominio: 2            | 82 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 82 Dominio: 5 |
| 38 Dominio: 3                         | 38 Dominio: 3            | 83 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 83 Dominio: 2 |
| 40 Dominio: 4                         | 40 Dominio: 4            | 84 Dominio: 8                         | 84 Dominio: 8 |
| 41 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 41 Dominio: 2            | 85 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 85 Dominio: 6 |
| 42 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 42 Dominio: 6            | 86 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 86 Dominio: 1 |
| 43 Dominio: 8                         | 43 Dominio: 8            | 87 Dominio: 7                         | 87 Dominio: 7 |
| 44 Dominio: 1, 2, 3, 4, 5, 6, 7, 8, 9 | 44 Dominio: 5            | 88 Dominio: 9                         | 88 Dominio: 9 |

Figura 08: Dominios antes y después de AC3 1/2

Figura 09: Dominios antes y después de AC3 2/2

- En los **sudokus difíciles (m3, m4)**, FC destaca claramente por su **poda anticipada**, reduciendo más del **40 % del tiempo** frente a BK.
- En el **caso inconsistente (m5)**, ambos fallan, pero FC **detecta antes el conflicto**, y en el **caso vacío (m6)** las diferencias son mínimas.

Por su parte, **AC3** mejora el rendimiento de forma variable según la dificultad del tablero:

- En **m1 y m2** apenas influye, mientras que en **m3 y m4** tiene un **efecto muy positivo**, reduciendo tiempos (hasta un **90 %**) y mejorando especialmente a FC.
- En **m5**, aunque no detecta la inconsistencia, **acelera la búsqueda posterior**, y en **m6** no aporta mejoras al no haber restricciones que propagar.

## Utilización de gráficas para el análisis de tiempos

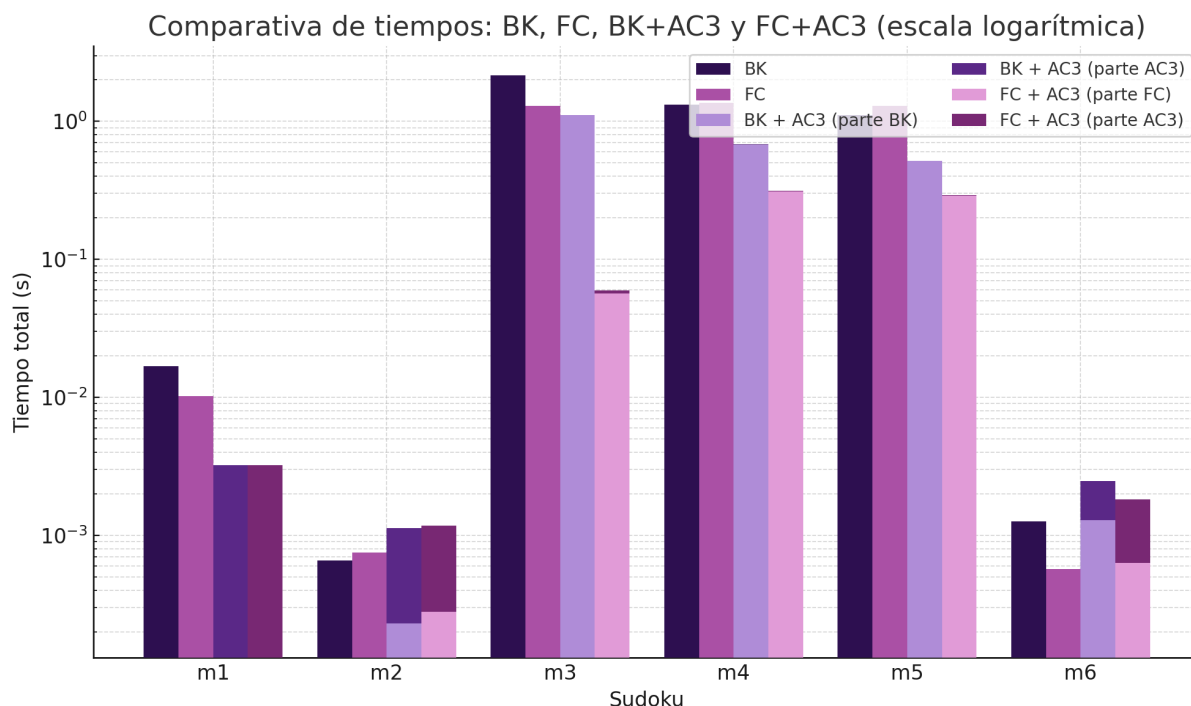


Figura 8: Gráfica de Comparativa Tiempos de Algoritmos

Como la gráfica usa una **escala logarítmica**, se suavizan visualmente las diferencias, pero los contrastes de tiempo entre los distintos algoritmos de cada sudoku son en realidad **enormes** en los casos más complejos (*m3* y *m4*).

Por otro lado, en los sudokus más sencillos (*m1* y *m2*), se aprecia que el AC3 reduce los tiempos de FC y BK considerablemente pero el **tiempo empleado por AC3** representa la **mayor parte del total**, sin embargo, a medida que **aumenta la dificultad**, esta proporción se invierte: el coste de AC3 pasa a ser **mínimo** frente al tiempo total de ejecución, además de que su efecto en la **aceleración de BK y sobretodo en FC** se vuelve cada vez más evidente.

Y como podemos observar en la siguiente Figura 9 ocurre algo similar con las recursiones, aunque se puede observar como BK (excepto en *m1* puesto que como vimos anteriormente, AC3 llega a resolver todo el sudoku) no se benefician recursión del recorte de AC3 mientras que con FC ocurre todo lo contrario, esto nos demuestra que la aplicación de AC3 reduce parte del espacio de búsqueda pero **no garantiza** que se reduzca el tamaño del árbol de búsqueda de BK en cuanto a recursiones.

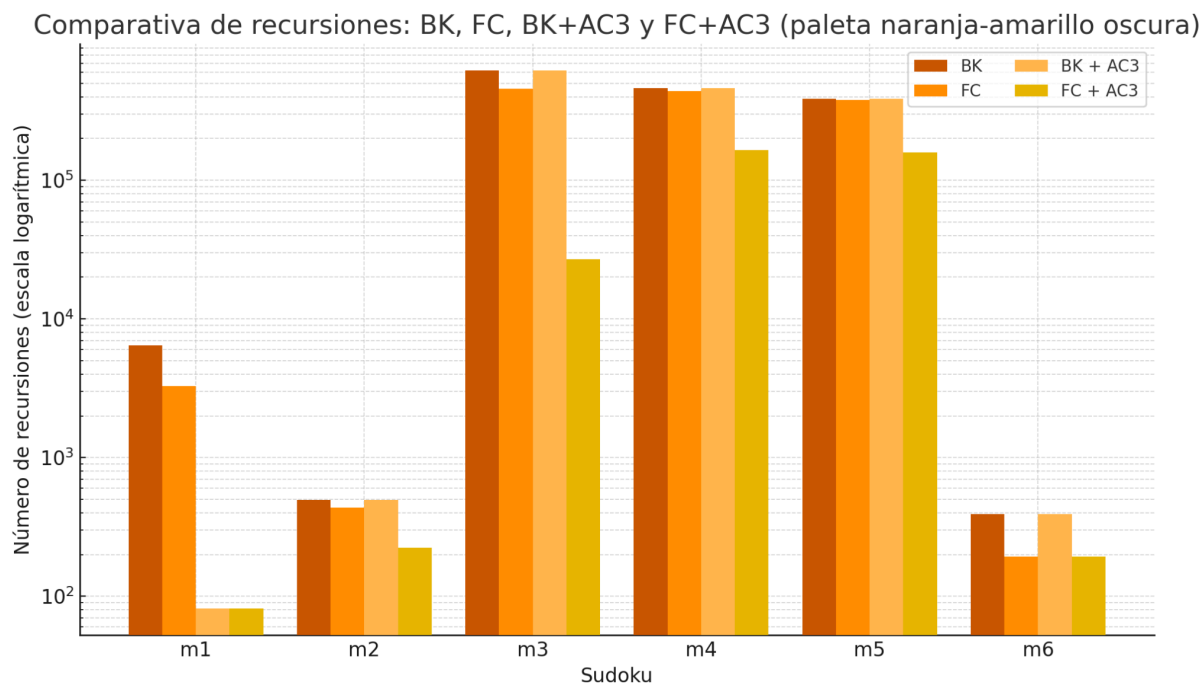


Figura 9: Gráfica de Comparativa Recursiones de Algoritmos

Y en el caso de las asignaciones que se ve ejemplificada en la Figura 10, podemos observar que AC3 sí reduce bastante las asignaciones en ambos algoritmos pero la que vuelve a salir más beneficiada es la del FC:

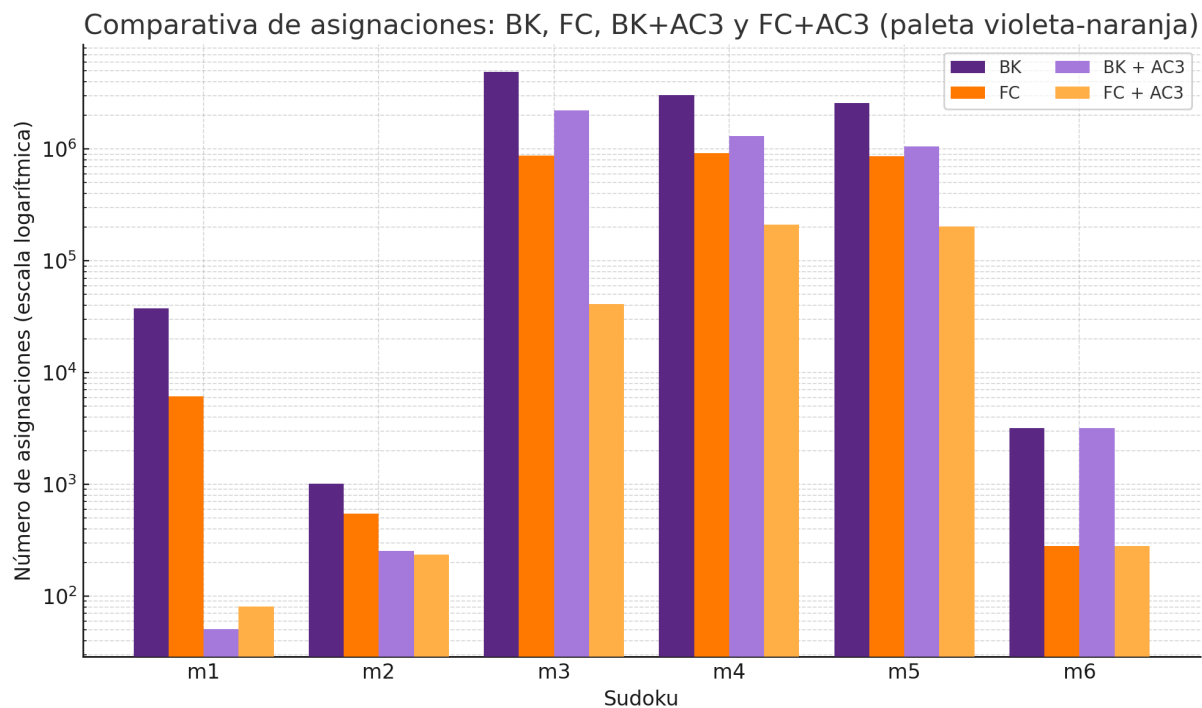


Figura 10: Gráfica de Comparativa Asignaciones de Algoritmos

## Dificultades encontradas

Durante el desarrollo del proyecto, la mayor parte de las dificultades estuvieron relacionadas con **trasladar la teoría a la práctica**. Al principio resultó complicado pasar del pseudocódigo teórico de los algoritmos (Backtracking, Forward Checking y AC3) a su implementación real, entendiendo qué debía ocurrir en cada paso y cómo se relacionaban los distintos procesos de asignación, poda y retroceso.

También fue complicado **entender y abstraer el paso a paso de cada algoritmo**, es decir, visualizar qué estaba ocurriendo en cada momento durante la ejecución. Resultaba difícil traducir esa secuencia lógica del pseudocódigo a un comportamiento real en el programa, especialmente al gestionar las asignaciones, las podas y los retrocesos dentro del proceso de resolución.

Durante la fase de pruebas, me encontré con que el **Forward Checking** tardaba más que el **Backtracking**, cuando debería ser al revés. El problema no estaba en restaurar los dominios, sino en la **estructura del filtrado**: el código original revisaba todas las variables y hacía demasiadas comprobaciones.

Después cambié el enfoque para usar la **lista de vecinos** y guardar solo los **cambios realizados en cada paso**, restaurando únicamente lo necesario. Con esta mejora el algoritmo se volvió mucho más eficiente y los resultados pasaron a ser los esperados.

Por último, al inicio también hubo confusión con el **retroceso del Backtracking**, ya que se implementó volviendo a llamar recursivamente a la función con  $k - 1$ , lo que generaba una recursión muy profunda y casi lineal. Cuestión que no me di cuenta hasta revisarla con el profesor.

## Uso de IAs generativas: explicación del uso que se ha hecho de ellas

Al principio, la usé para **crear las primeras versiones de los algoritmos** de Backtracking, Forward Checking y AC3, con el objetivo de entender de forma general cómo funcionaban y cuál debía ser su comportamiento. Aunque esas versiones iniciales no eran las que se pedían en la práctica, me sirvieron para **comprender la estructura básica y el flujo lógico** de cada uno.

A partir de ahí, comencé a profundizar en el **pseudocódigo teórico** y en su **traducción a Python**, utilizando ChatGPT como apoyo en cada paso. Lo empleé para **aclarar conceptos, detectar inconsistencias y contrastar mi implementación con la teoría, así como para darme nuevas ideas y perspectivas**, de forma que el resultado fuera lo más fiel posible al planteamiento formal del problema.

En una fase más avanzada también utilicé **QWEN3 en su versión razonadora**, en un punto en el que ni ChatGPT ni yo conseguíamos reducir los tiempos de ejecución del **Forward Checking**. Esta herramienta me ofreció **una perspectiva totalmente diferente**, ayudándome a ver que el problema no estaba en la lógica del código, sino en **la forma de tratar los datos**. Gracias a esa visión, pude **replantear la estructura del algoritmo y mejorar su rendimiento**.

Finalmente, también he usado ChatGPT en la fase de **documentación**, no para escribir todo el contenido, sino para **asegurar la coherencia y consistencia** de las explicaciones. Me ha servido para **pulir la redacción y verificar que las ideas estuvieran bien estructuradas**, siempre desde **mi propia perspectiva y razonamiento personal**, y sabiendo que, a diferencia del código, donde el ordenador ofrece un *feedback* inmediato, en este caso **una IA tiende a dar la razón incluso cuando existen errores**, por lo que cualquier apartado es **susceptible de contener imprecisiones conceptuales que provienen de mí**, ya sea por desconocimiento o por interpretación errónea de algunos conceptos.

## Apartado de referencias bibliográficas

- [Sudoku Solver by Andrew Stuart](#)
- [Explicando Backtracking | Fuerza Bruta Pero Elegante - YouTube](#)<sup>2</sup>
- [Let's make a sudoku solver in 5 minutes \(Backtracking\) - Inside code - YouTube](#)<sup>2</sup>

---

<sup>2</sup>Estas fuentes se han empleado para **comprender el concepto de Backtracking desde una perspectiva general y teórica**, sirviendo como base para su posterior implementación y análisis en el contexto del problema del Sudoku